STAJ PROJESİ · TÜRKSAT AVIATION AND COMMUNICATION

Akıllı Okul Bilgi Sistemi

Hafta 5 — Niyet Analizi, Açıklanabilirlik ve Sertleştirme

HAZIRLAYAN	Bera Ergin — s29315
KURUM	PJAIT — 7. Yarıyıl
BU HAFTA	Uçtan uca akışı olgunlaştırma ve kötüye kullanıma karşı koruma

Nereden Geliyoruz, Nereye Gidiyoruz

HAFTA 4 — BİTTİ

- ✓ POST /query çalışıyor
- ✓ Claude ile doğal dil → SQL
- ✓ RuleEngine her sorguyu sarmalıyor
- ✓ Ama her soru SQL sorgusu sayılıyor

"Merhaba" yazan bir kullanıcı bile Claude'a SQL ürettirmeye zorlanıyor — hem gereksiz hem kırılgan.

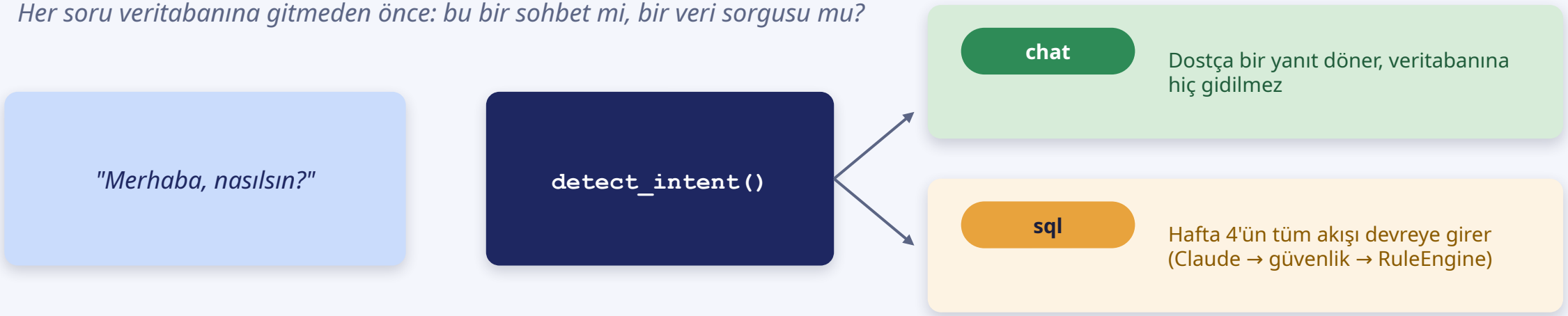
HAFTA 5 — HEDEF

- Niyet analizi — sohbet mi, sorgu mu?
- Sonuçla birlikte kısa bir açıklama (XAI)
- Rate limiting — /query'yi kötüye kullanıma kapatma
- Yapılandırılmış (JSON) loglama

Yeni bir endpoint yok — mevcut /query olgunlaşıyor.

Niyet Analizi

Her soru veritabanına gitmeden önce: bu bir sohbet mi, bir veri sorgusu mu?



Neden test edilebilir tasarladık

detect_intent(question, client=None) — client parametresi opsiyonel. Testlerde sahte (mock) bir istemci veriliyor, böylece Claude'a gerçek bir API çağrısı yapmadan 'chat' ve 'sql' dallarının doğru çalıştığı doğrulanabiliyor. Gerçek çalışma zamanında client verilmezse, gerçek Anthropic istemcisi kullanılıyor. Bu, bağımlılık enjeksiyonu (dependency injection) olarak bilinen, API anahtarı olmadan da birim testi yazılabilmesini sağlayan bir tasarım deseni.

Açıklanabilirlik (Explainable AI)

Sonuç yalnızca sayı değil — neden bu sonucun döndüğünü de anlatıyor

Hafta 4 — yalnızca veri

```
{
  "row_count": 4,
  "results": [...]
}
```

Kullanıcı 4 sayısını görüyor ama neden 4 olduğunu bilmiyor.

Hafta 5 — veri + açıklama

```
{
  "row_count": 4,
  "results": [...],
  "explanation": "9-A sınıfı
   öğretmeni olduğunuz için
   yalnızca bu sınıftaki 4
   öğrenci gösterildi."
}
```

Açıklama metni LLM'e değil, RuleEngine'in uyguladığı filtreye dayanıyor (kural tabanlı, deterministik) — Claude'un sonucu "açıklarken" uydurma (hallucination) riski taşımasını önlemek için bilinçli bir tercih.

Rate Limiting

Her Claude çağrısının bir maliyeti var — /query'yi aşırı kullanıma kapatmak gerekiyor

Pencere	Limit	Depolama	Aşım durumunda
60 saniyelik kayan pencere (sliding window)	Kullanıcı başına dakikada 10 istek	Bellek içi (in-memory) dict — kalıcı değil	429 Too Many Requests + Retry-After header

Bilinen sınırlama (bilinçli olarak belgelenmiştir)

Bellek içi depolama, sunucu birden fazla process/worker ile çalıştığında (ör. birden fazla uvicorn worker'ı) her worker'ın kendi sayacını tutması anlamına gelir — limit gerçekte N × worker_sayısı olur. Tek worker'lı geliştirme/demo ortamı için yeterli; production'da Redis gibi paylaşılan bir depoya geçiş gerekir. Bu sınırlama README'de açıkça belirtilir.

Yapılandırılmış Loglama

Her /query isteği, tek satırlık bir JSON log kaydı bırakır

```
{
  "timestamp": "2026-10-12T14:32:01Z",
  "user": "teacher1",
  "role": "teacher",
  "intent": "sql",
  "question": "Sınıfımdaki öğrenci sayısı kaç?",
  "row_count": 4,
  "duration_ms": 842,
  "rate_limited": false
}
```

Üretilen ham SQL loglanır ama kullanıcı sorusunun/sonucun kendisi hassas veri içerebileceğinden, bu log yalnızca sunucu tarafında tutulur — hiçbir zaman API yanıtına veya istemci tarafına sızmaz.

Bu Hafta Bağımlılık Eklenmedi

Bilinçli bir tercih — standart kütüphane yeterliydi

Rate limiting	Python'ın yerleşik collections.deque ve time modülleriyle yazıldı — slowapi gibi bir kütüphaneye gerek duyulmadı
Loglama	Python'ın yerleşik logging modülü, JSON formatlı bir handler ile yapılandırıldı
Niyet analizi	Mevcut anthropic SDK'sı (Hafta 4'ten) yeniden kullanıldı, yeni bir paket eklenmedi

requirements.txt Hafta 4'ten değişmedi: fastapi, uvicorn, python-jose, passlib, sqlalchemy, anthropic

Hafta 5 — Kapsam ve Kabul Kriterleri

Bu hafta teslim edilenler

- ✓ backend/app/services/intent_service.py
- ✓ backend/app/services/explain_service.py
- ✓ backend/app/core/rate_limiter.py
- ✓ backend/app/core/logging_config.py
- ✓ query.py'ye niyet/açıklama/limit entegrasyonu
- ✓ Üç servis için bağımsız birim testleri

Kabul kriterleri

1. Sohbet niyeti veritabanına hiç gitmeden yanıtlanır
2. SQL niyeti Hafta 4'ün tüm akışından geçer
3. 11. istekte (dakikada) 429 döner, Retry-After header'ı vardır
4. Her /query isteği tek satır JSON log üretir
5. intent_service ve rate_limiter testleri API key olmadan geçer

Sonraki Adım — Hafta 6

Streamlit Arayüzü

- Giriş ekranı ve sorgu sekmesi
- TR/EN dil seçimi
- API'ye bağlanan ilk görsel arayüz
- Artık kullanıcılar curl yerine gerçek bir ekran görecek

Teşekkürler